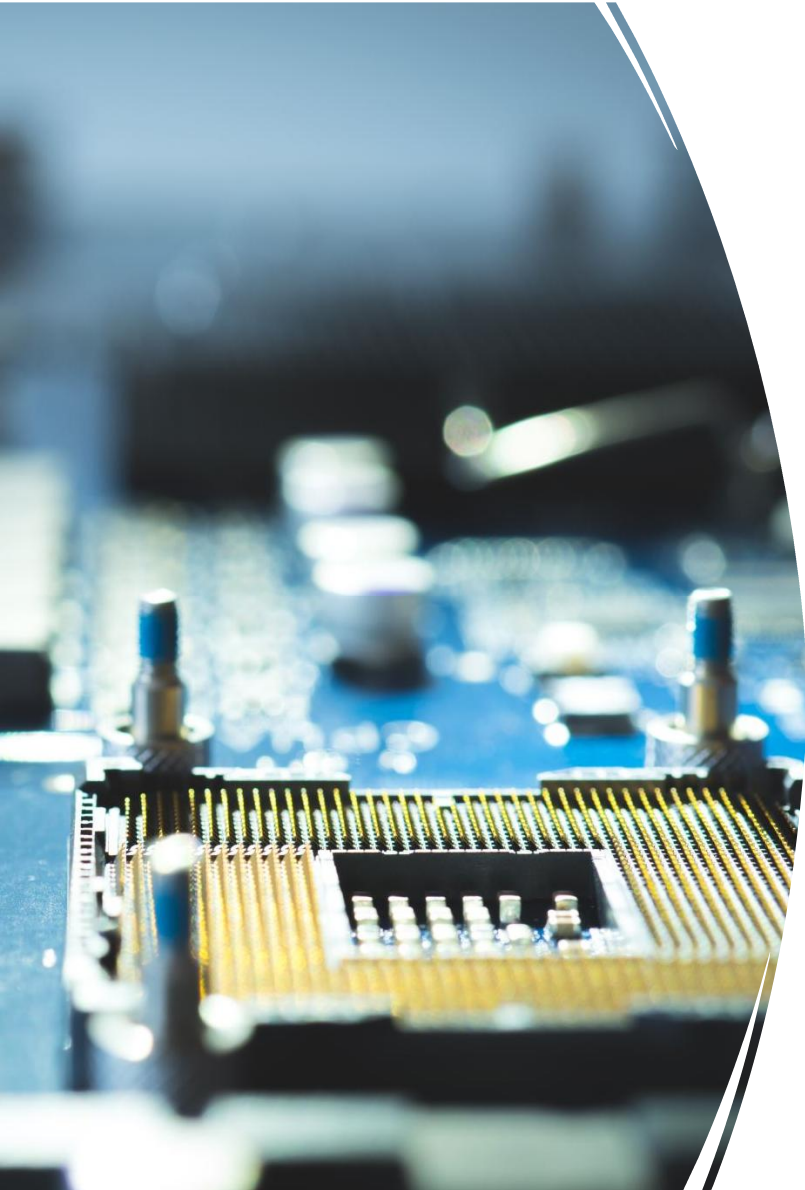


Airflow



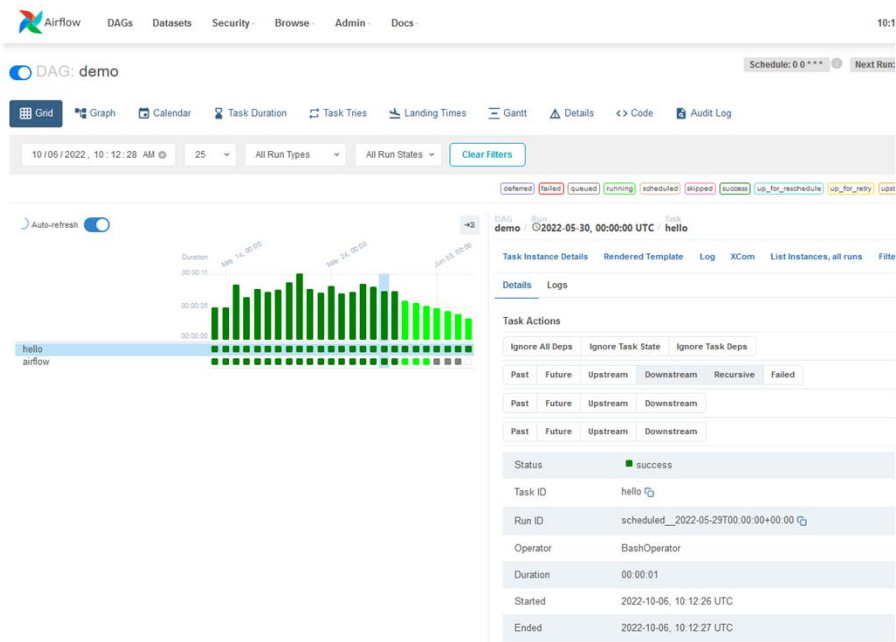
- Open Source – Criado pelo Airbnb em 2015. Mantido pela fundação Apache
- Desenvolvido em Python
- Extensível



Orquestrador

- Airflow não processa dados, ele coordena o processamento
- O processamento ocorre em:
 - Sistema Operacional
 - Banco de Dados
 - Spark
 - Elastic Search
 - Etc.
- Em batch

Interfaces



- Web
- CLI

DAGs



```
from datetime import datetime

from airflow import DAG
from airflow.decorators import task
from airflow.operators.bash import BashOperator

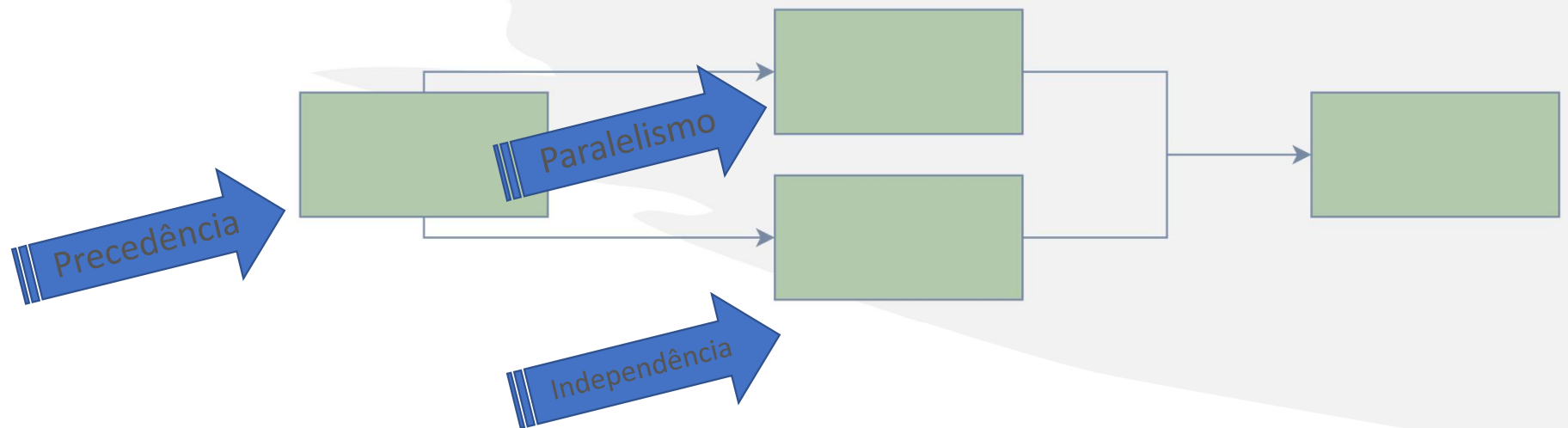
# A DAG represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2022, 1, 1), schedule="0 0 * * *") as dag:

    # Tasks are represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo hello")

    @task()
    def airflow():
        print("airflow")

    # Set dependencies between tasks
    hello >> airflow()
```

DAG: Pipeline



Pipeline



Airflow

- DAG: é um pipeline do Airflow
 - O que este pipeline vai fazer?
- Operators/Task



DAG

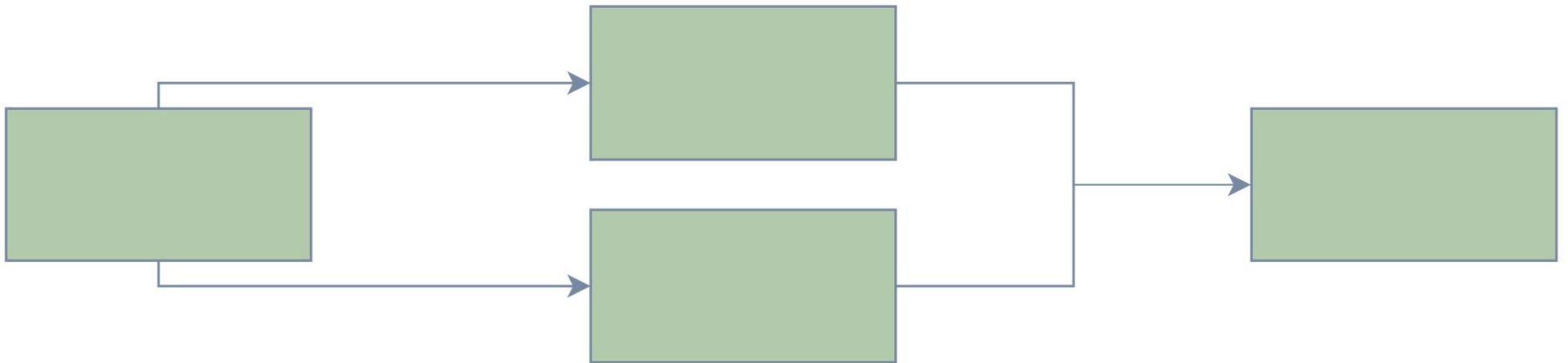
**Directed acyclic
graph: Grafo
direcionado
acíclico**

**Grafo: Conjunto
de Vértices e
Arestas**

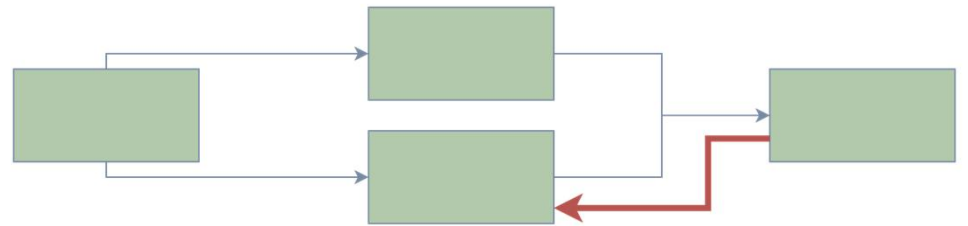
**Direcionado:
Possui Direção
Específica**

**Acíclico: Não
Circular**

DAG



Sem Ciclos



Airflow

DAG é um script python

Não precisa ter a lógica (classe externa)

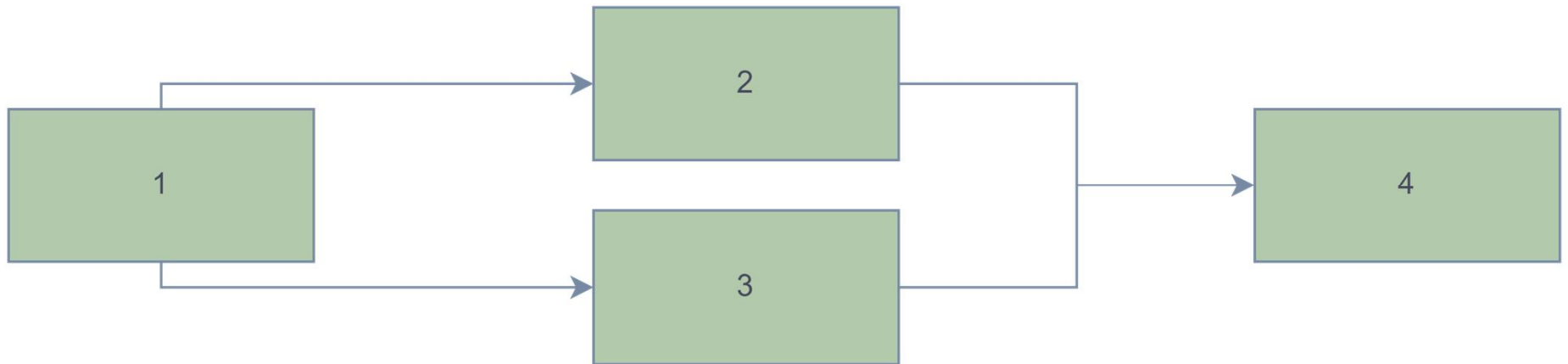
Um mesmo script pode ter múltiplas DAGs

Uma DAG pode ter uma ou mais tasks

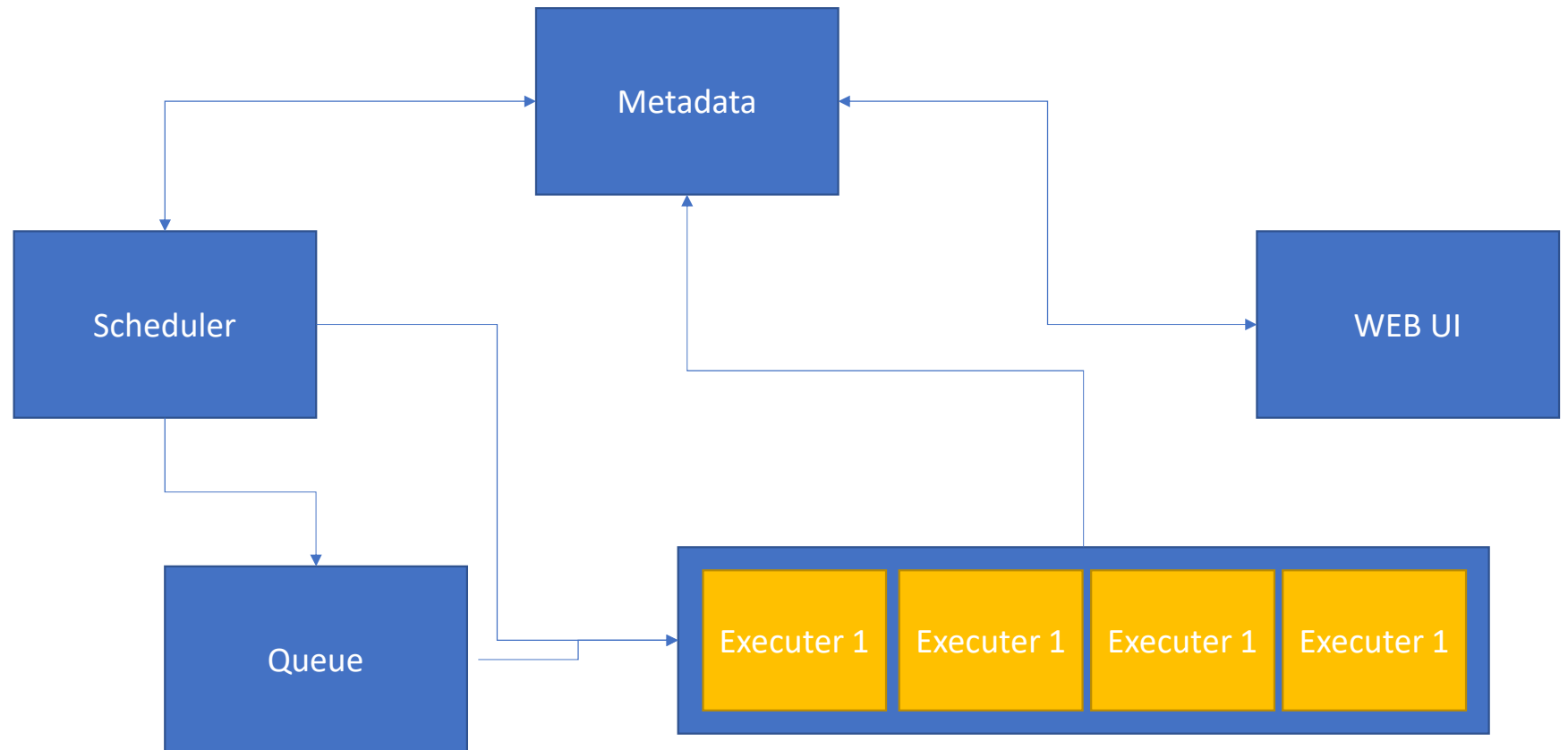
Tasks possuem dependência

DAG

- Operators: O que é de fato executado.
- Tasks: instância de um operador



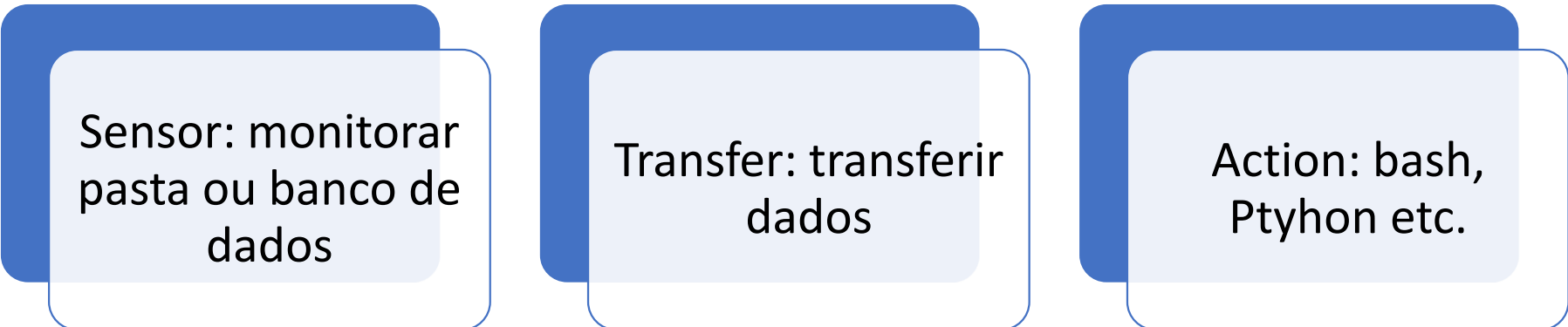
Arquitetura



Operators

- Algo a ser feito no pipeline
- Uma vez instanciado é uma tarefa
- Existem muitos tipos, derivam de BaseOperator

Tipos de Operators



Sensor: monitorar
pasta ou banco de
dados

Transfer: transferir
dados

Action: bash,
Python etc.

Providers

- Interação com outras ferramentas:
 - Aws
 - Databricks
 - MySQL
 - MongoDB

Regras de Gatilho

all_success

all_failed

all_done

one_failed

one_success

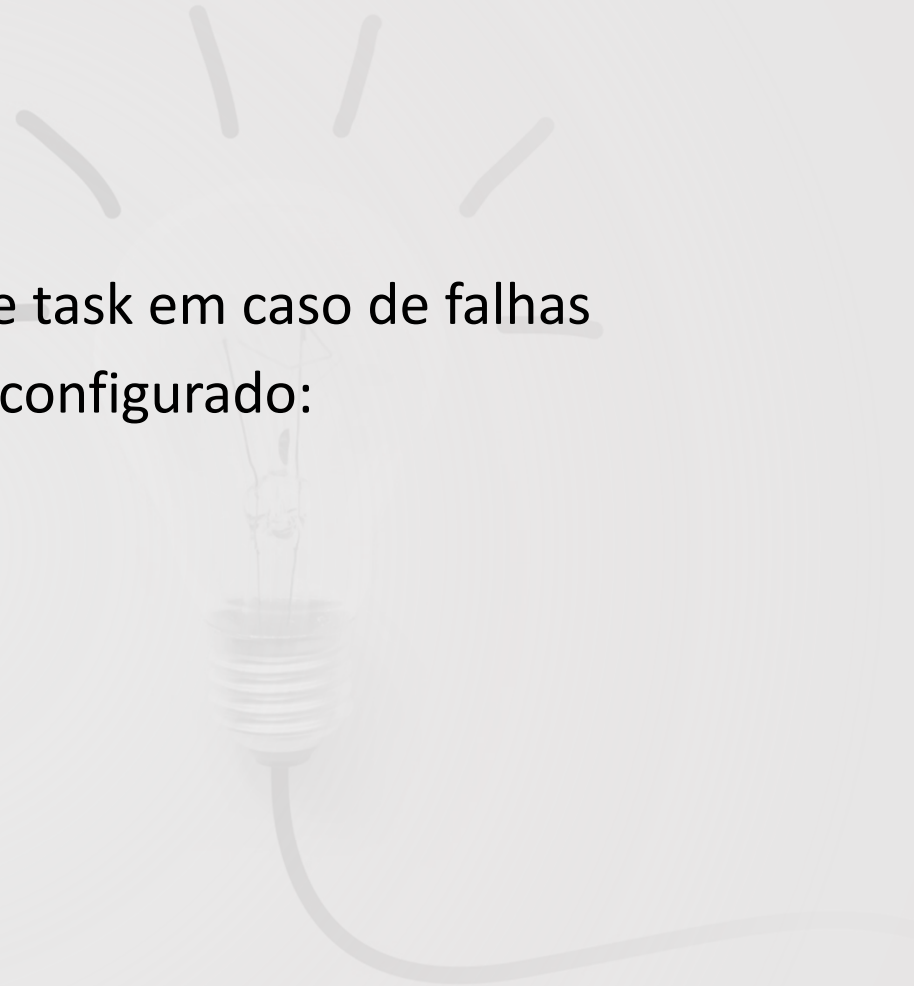
none_failed

none_skipped

dummy

Gerenciamento de Falhas

- Permite novas tentativas de execução de task em caso de falhas
- O número de novas tentativas pode ser configurado:
 - Globalmente (**airflow.cfg**)
 - Na Dag
 - Na Task





Hooks

- Abstração para interagir com sistemas externos: Exemplo: **PostgresHook**